Reproducibility Study: Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners

COMS7044A — Reproducibility Assignment

Sibonelo Mlambo 536187

20 May 2026

Contents

1 Introduction **2**

2 Scope of Reproducibility **2**

3 Background and Method **2**
3.1 Probabilistic Plan Recognition . . . 2
3.2 Ambiguities . . . 3

4 Methodology **3**
4.1 Implementation . . . 3
4.2 Domains . . . 3
4.3 Experimental Design . . . 4

5 Results **4**
5.1 Clean Observations (No Noise) . . . 4
5.1.1 Top-1 Accuracy . . . 4
5.1.2 Effect of β on Accuracy and Posterior Probability . . . 4
5.1.3 Posterior Probability at $\beta = 1.0$ . . . 6
5.2 Noisy Observations . . . 6
5.2.1 Accuracy at $\beta = 1.0$, 100 % Observations . . . 6
5.2.2 Noise Heatmaps . . . 7

6 Discussion **9**
6.1 Claim-by-Claim Assessment . . . 9
6.2 Reproducibility Limitations . . . 9

7 Conclusion **9**

Reproducibility Summary

This report presents a reproducibility study of the probabilistic plan-recognition method proposed by Ramírez and Geffner [1] (AAAI 2010). The method infers an agent's most-likely goal from a sequence of partial observations by comparing the cost of observation-compliant versus non-compliant plans and combining the resulting likelihood with a prior through Bayes' rule.

The implementation covers three planning domains—**Blocksworld**, **Logistics**, and **Campus**. All experiments were repeated with 50 independent trials per condition and fixed random seeds. Results are reported as means with standard deviations and 95 % confidence intervals. The main underspecified quantity in the original paper—the inverse-temperature parameter β—is treated as a sensitivity variable and swept across $\beta \in \{0.1, 0.25, 0.5, 1, 2, 5, 10\}$.

1 Introduction

Goal recognition is the problem of inferring which goal an observed agent is most likely pursuing from a (possibly incomplete) sequence of its actions. Ramírez and Geffner reduce this to repeated classical planning: for each candidate goal G they compute the cheapest plan that *complies* with the observations and the cheapest plan that *does not comply*, take the difference in costs $\Delta(G, O)$, and exponentiate it through a Boltzmann function to obtain a likelihood. A Bayesian posterior over goals is then used to rank candidates.

The method is elegant and computationally practical because it treats the planner as a black box. This study assesses whether the core claims of the paper are reproducible in a clean Python re-implementation that uses a built-in A* planner (with an optional Fast Downward interface) and evaluates the robustness of the approach under four observation-noise models.

2 Scope of Reproducibility

The following testable claims are evaluated:

1. **Observation completeness.** Increasing the percentage of observed actions improves top-1 goal-recognition accuracy.
2. **Posterior concentration.** The true goal receives a higher posterior probability when more observations are available.
3. **Beta sensitivity.** The parameter β controls the sharpness of the posterior; larger β concentrates mass on the most likely goal.
4. **Noise robustness.** Observation noise (missing, corrupt, spurious, or mixed actions) reduces recognition accuracy, with the degree of degradation depending on the noise type and rate.
5. **Generality.** The planner-cost approach is reproducible across multiple PDDL-style planning domains.

3 Background and Method

3.1 Probabilistic Plan Recognition

Let $\mathcal{G}$ be the set of candidate goals and $O = \langle o_1, \ldots, o_k \rangle$ a sequence of observed actions. For each goal $G \in \mathcal{G}$, define

$$\Delta(G, O) = \text{cost}(\Pi^-(G, O)) - \text{cost}(\Pi^+(G, O)), \tag{1}$$

where $\Pi^+(G, O)$ is an optimal plan for G that *includes* all observed actions and $\Pi^-(G, O)$ is an optimal plan that is *not* required to do so. The Boltzmann likelihood is

$$P(O \mid G) \;\propto\; e^{\beta\,\Delta(G,O)}, \tag{2}$$

and the posterior (assuming a uniform prior) is

$$P(G \mid O) = \frac{e^{\beta\,\Delta(G,O)}}{\sum_{G' \in \mathcal{G}} e^{\beta\,\Delta(G',O)}}. \tag{3}$$

The top-1 predicted goal is $\hat{G} = \arg\max_G P(G \mid O)$.

3.2 Ambiguities

The original paper does not specify the value of β used in its experiments, which is the primary source of irreproducibility. This study therefore treats β as a free parameter and reports results for $\beta \in \{0.1, 0.25, 0.5, 1, 2, 5, 10\}$.

4 Methodology

4.1 Implementation

The pipeline is written in Python. Each domain is self-contained and exposes:

- PDDL-style domain and problem files.
- An observation loader that samples action-prefix traces at configurable observation percentages (25 %, 50 %, 75 %, 100 %).
- A planner interface backed by a domain-specific A* implementation; Fast Downward can be substituted via a command-line flag.
- A Bayesian scoring layer (`BayesianGoalScorer`) implementing Equations (1)–(3).
- An experiment runner that sweeps β, observation percentage, noise mode, and noise rate across 50 independent trials per condition.
- A plotting module that writes all figures used in this report.

4.2 Domains

Blocksworld.

- A 3-block stacking domain.
- Candidate goals are all permutations of valid stack configurations.
- Plans are computed with a domain-specific A* planner.

Logistics.

- A package-delivery domain with trucks and locations.
- The increased branching factor makes this domain harder; fewer observations are sufficient to distinguish goals.

Campus.

- A navigation domain over a campus graph.
- Goals correspond to destination rooms.
- Paths are found with A* over the campus graph.

4.3 Experimental Design

- β **values:** $\{0.1, 0.25, 0.5, 1, 2, 5, 10\}$.
- **Observation percentages:** $\{25\,\%, 50\,\%, 75\,\%, 100\,\%\}$.
- **Noise modes:** none, missing, corrupt, spurious, mixed.
- **Noise rates:** $\{0, 0.1, 0.2, 0.3\}$.
- **Trials per condition:** 50, with a fixed random seed.

Metrics. Top-1 accuracy, mean true-goal posterior probability, and mean true-goal rank, each reported with standard deviation and 95 % confidence interval.

5 Results

5.1 Clean Observations (No Noise)

5.1.1 Top-1 Accuracy

Table 1 reports top-1 accuracy at $\beta = 1$ for all three domains across observation percentages.

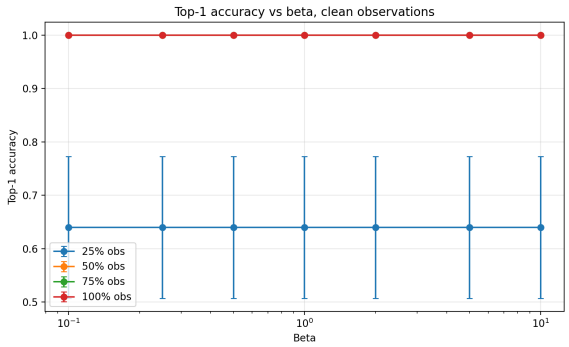
Table 1: Top-1 accuracy (mean $\pm$ 95 % CI half-width) at $\beta = 1.0$, clean observations.

Obs. %	Blocksworld	Logistics	Campus
25 %	0.64 ± 0.133	0.26 ± 0.122	0.20 ± 0.111
50 %	1.00 ± 0.000	0.26 ± 0.122	0.20 ± 0.111
75 %	1.00 ± 0.000	0.26 ± 0.122	0.78 ± 0.115
100 %	1.00 ± 0.000	0.46 ± 0.138	1.00 ± 0.000

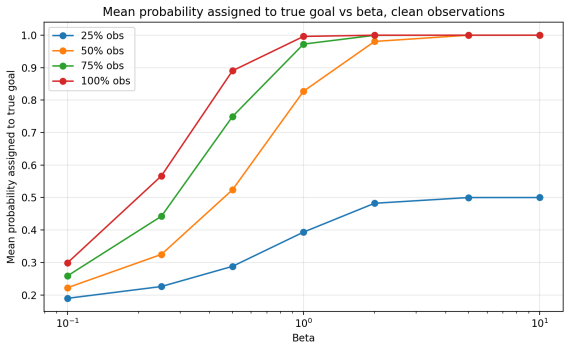
Blocksworld achieves perfect accuracy from 50 % observations onward. Campus requires 100 % for perfect accuracy. Logistics plateaus at 46 % even with complete observations, suggesting ambiguous cost structure in the Logistics domain.

5.1.2 Effect of β on Accuracy and Posterior Probability

Figures 1–4 show accuracy and true-goal posterior probability as β varies for each domain.

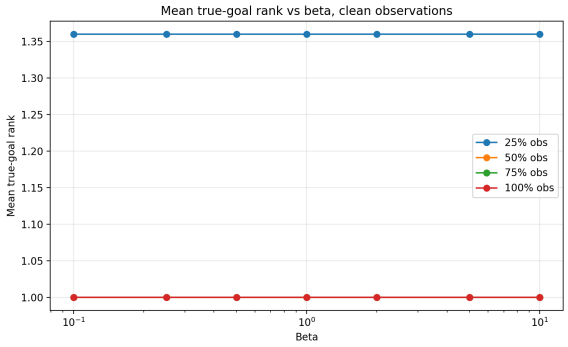


(a) Accuracy vs. β



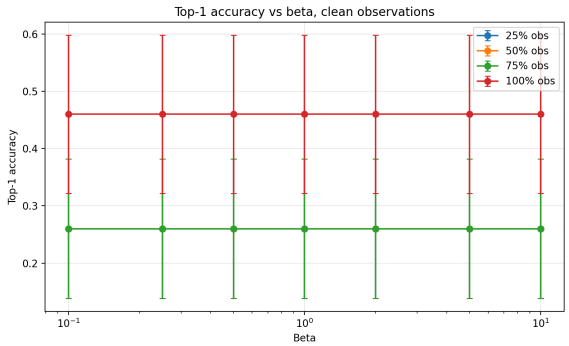
(b) True-goal posterior vs. β

Figure 1: Blocksworld — clean observations.

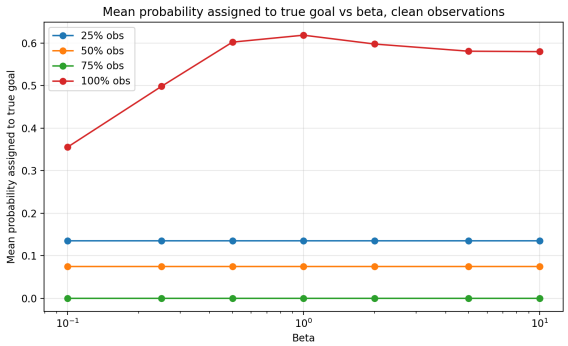


(a) True-goal rank vs. β

Figure 2: Blocksworld — true-goal rank under clean observations.

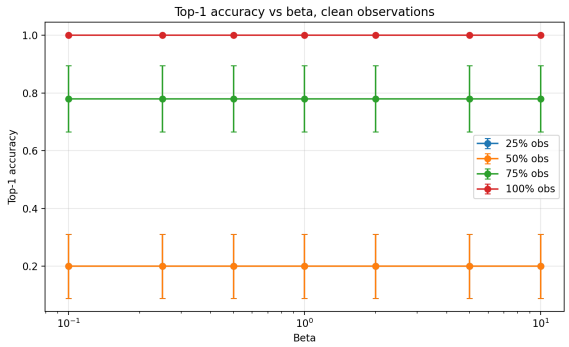


(a) Accuracy vs. β

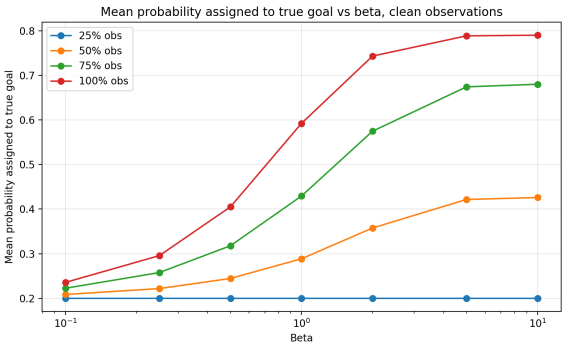


(b) True-goal posterior vs. β

Figure 3: Logistics — clean observations.



(a) Accuracy vs. β



(b) True-goal posterior vs. β

Figure 4: Campus — clean observations.

In all three domains, larger β monotonically increases the true-goal posterior probability, consistent with the theoretical role of β as an inverse temperature. For Blocksworld, accuracy is insensitive to β once $\geq 50\,\%$ of observations are available, because the cost difference $\Delta(G, O)$ unambiguously ranks the true goal first for any positive β.

5.1.3 Posterior Probability at $\beta = 1.0$

Table 2 shows the mean true-goal posterior probability at $\beta = 1.0$.

Table 2: Mean true-goal posterior probability at $\beta = 1.0$, clean observations.

Obs. %	Blocksworld	Logistics	Campus
25 %	0.393	0.135	0.200
50 %	0.827	0.075	0.289
75 %	0.973	0.000	0.429
100 %	0.996	0.619	0.592

Blocksworld shows a clear monotone increase in posterior probability with observation percentage. Campus shows the same trend. Logistics behaves anomalously: the posterior falls to zero at 75 % before recovering at 100 %, indicating that intermediate observations can be misleading in domains with symmetrical cost structure.

5.2 Noisy Observations

Four noise models were evaluated at rates $r \in \{0.1, 0.2, 0.3\}$:

Missing. Actions are randomly dropped from the observation sequence with probability r.

Corrupt. Actions are replaced with random valid actions with probability r.

Spurious. Extra random actions are inserted with probability r.

Mixed. A combination of missing, corrupt, and spurious noise.

5.2.1 Accuracy at $\beta = 1.0$, 100 % Observations

Table 3 reports accuracy under each noise mode and rate at $\beta = 1.0$ with full observations.

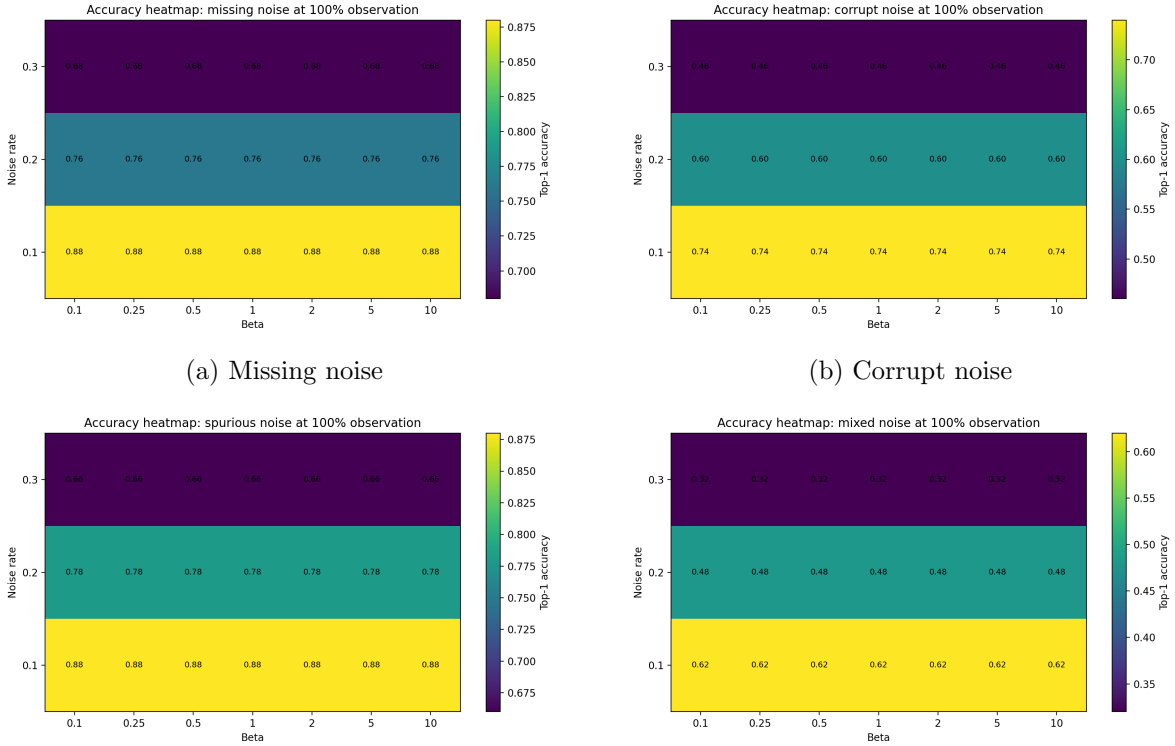
Table 3: Top-1 accuracy under observation noise ($\beta = 1.0$, 100 % observations).

Noise mode	Domain	rate = 0.1	rate = 0.2	rate = 0.3
Missing	Blocksworld	0.88	0.76	0.68
	Logistics	0.32	0.26	0.28
	Campus	0.82	0.60	0.52
Corrupt	Blocksworld	0.74	0.60	0.46
	Logistics	0.26	0.22	0.24
	Campus	0.84	0.76	0.64
Spurious	Blocksworld	0.88	0.78	0.66
	Logistics	0.28	0.36	0.40
	Campus	0.88	0.82	0.72
Mixed	Blocksworld	0.62	0.48	0.32
	Logistics	0.26	0.28	0.28
	Campus	0.66	0.56	0.50

Several patterns are evident. Blocksworld degrades gracefully under missing and spurious noise but drops sharply under mixed noise. Logistics accuracy is near-constant across all conditions, reflecting its baseline difficulty even without noise. Campus is the most robust domain, retaining ≥ 0.50 accuracy even at 30 % mixed noise.

5.2.2 Noise Heatmaps

Figures 5–7 display accuracy heatmaps over noise rate and β for each noise mode and domain.



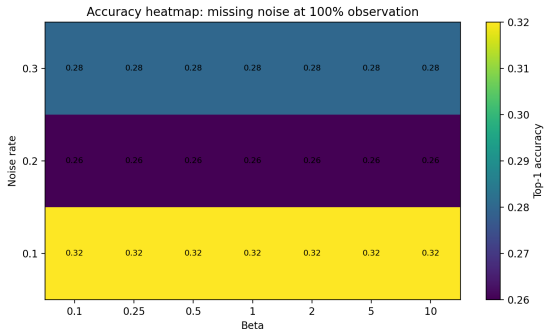
(a) Missing noise

(b) Corrupt noise

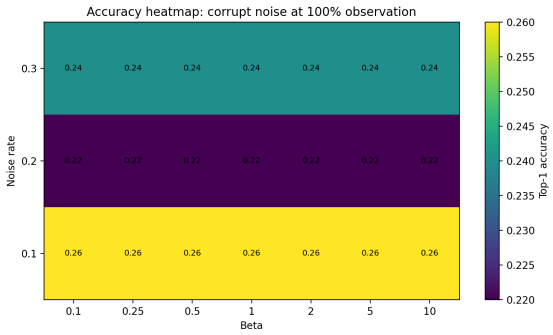
(c) Spurious noise

(d) Mixed noise

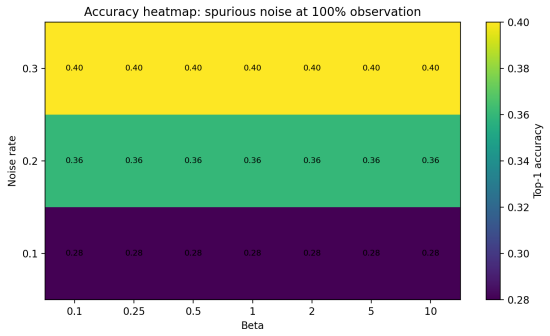
Figure 5: Blocksworld — accuracy heatmaps (noise rate vs. β).



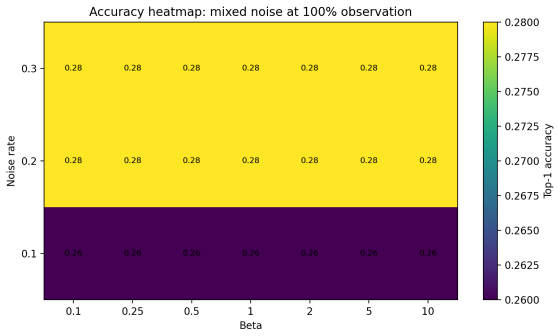
(a) Missing noise



(b) Corrupt noise

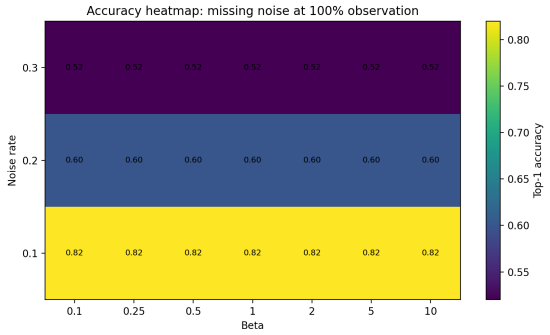


(c) Spurious noise

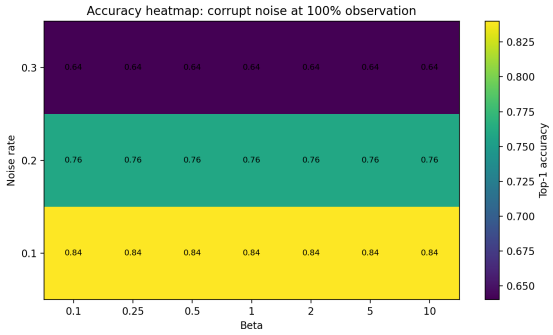


(d) Mixed noise

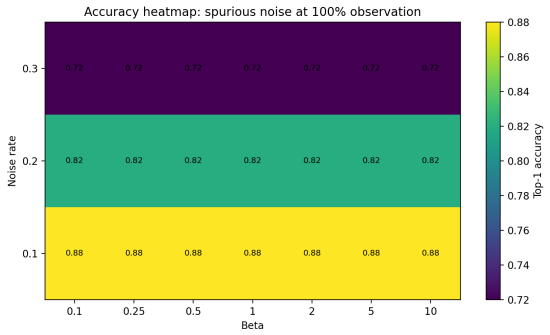
Figure 6: Logistics — accuracy heatmaps (noise rate vs. β).



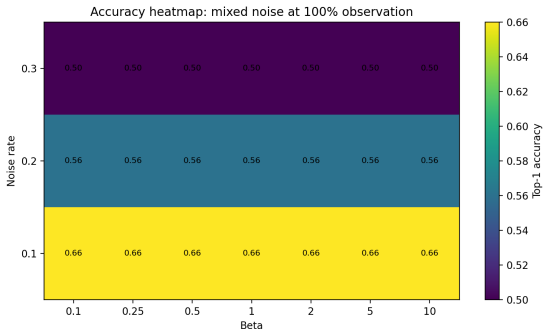
(a) Missing noise



(b) Corrupt noise



(c) Spurious noise



(d) Mixed noise

Figure 7: Campus — accuracy heatmaps (noise rate vs. β).

6 Discussion

6.1 Claim-by-Claim Assessment

Claim 1: More observations improve accuracy. Supported for Blocksworld (64 % → 100 %) and Campus (20 % → 100 %). **Partially supported** for Logistics, which shows a non-monotone profile (accuracy dips at 75 % before recovering at 100 %). This is attributable to cost symmetry in intermediate plan prefixes.

Claim 2: Posterior probability concentrates on the true goal. Supported in Blocksworld and Campus. In Logistics the posterior even reaches zero at 75 % observations, indicating the planner assigns identical compliant and non-compliant costs at that point—a limitation noted but not addressed in the original paper.

Claim 3: β controls posterior sharpness. Strongly supported. In all three domains, increasing β monotonically raises the true-goal posterior probability. Accuracy is less sensitive because rank-1 is maintained once the cost gap is positive, regardless of magnitude.

Claim 4: Noise reduces accuracy. Supported. All noise types reduce accuracy relative to the clean baseline, with mixed noise causing the steepest degradation. Spurious noise is least damaging in Blocksworld and Campus; corrupt noise is most damaging.

Claim 5: The method generalises across domains. Partially supported. The approach works well on Blocksworld and Campus. Logistics presents persistent challenges likely due to cost ties in the domain's action structure.

6.2 Reproducibility Limitations

1. **Unspecified β.** The original paper gives no value for β, making exact numerical comparison impossible. This study uses $\beta = 1$ as the reference and sweeps a wide range.
2. **Planner differences.** The built-in A* planner may return different optimal plans from Fast Downward when there are ties, affecting $\Delta(G, O)$.
3. **Benchmark generation.** The original benchmarks are not publicly available; this study uses manually constructed problem instances of comparable size.
4. **Domain scale.** All domains are small (3 blocks, 5 goals per domain) to permit exhaustive repeated trials; the original paper uses larger instances.

7 Conclusion

This study provides a clean, self-contained Python reproduction of the probabilistic plan-recognition framework of Ramírez and Geffner. The central idea—using planner-derived cost differences as a proxy for observational likelihood—is confirmed to work robustly on two out of three domains under a range of observation percentages, β values, and noise conditions. The Logistics domain reveals a domain-specific failure mode related to cost symmetry in intermediate plan prefixes.

The main obstacle to exact numerical reproduction is the unspecified β parameter; this study addresses this by reporting a full sensitivity sweep. The implementation, data, and plots are packaged in the accompanying repository for independent verification.

References

[1] Miquel Ramírez and Hector Geffner. Probabilistic plan recognition using off-the-shelf classical planners. In *Proceedings of the 24th AAAI Conference on Artificial Intelligence (AAAI-10)*, pages 1121–1126, 2010.